

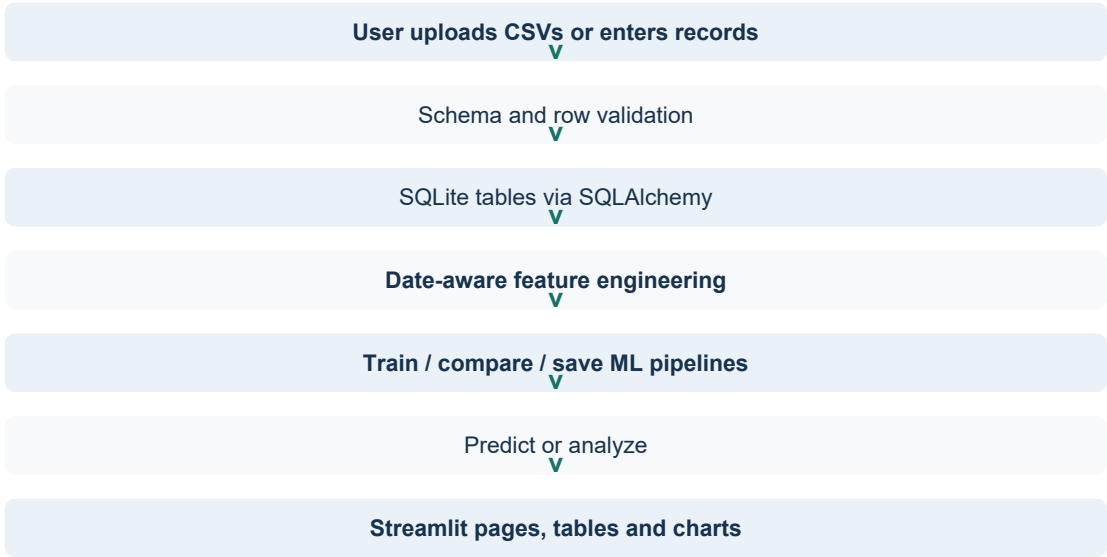
Football ML Monitoring System

A code-verified guide to the data, calculations, machine learning, database and user workflow.

THE PURPOSE

A Streamlit application for importing football-player records, viewing longitudinal information and producing separate injury-risk, next-match rating and documented health-event risk estimates. Health trends and clustering add descriptive analysis. It does not combine the three predictions into one score.

How the application works



The four analytical outputs

Output	What it means	Basis
Injury risk	Probability of recorded injury onset in a future 7-day window.	Supervised classification
Performance	Predicted rating for the next match record.	Supervised regression
Health risk	Probability of a documented health event in a future 7-day window.	Supervised classification
Trends / clusters	Longitudinal measurements; groups of similar profiles.	Statistics / unsupervised ML

Input: eight supported CSV datasets (or manual entry). Calculated: workload, recent recovery/match/health history and medical-test changes. Predicted: the three separate model outputs above; only after enough labelled records and explicit model training.

Technology and architecture

Technology is listed only when it appears in the current code, dependencies or Docker setup.

Technology	What it is	Why / where used
Python 3.12	Application language	Feature, validation, model and UI code; Docker base image is python:3.12-slim.
Streamlit	Python web UI framework	Sidebar pages, forms, uploaders, buttons, tables and charts in app.py / views/.
SQLite	File database	Persistent records in data/football_ml.db by default.
SQLAlchemy + SQL	ORM / database access	Table models, sessions, queries, foreign keys, transactions.
pandas	Tabular data library	CSV parsing, validation frames, feature frames, display tables.
NumPy	Numeric library	Slopes, statistics, arrays and model calculations.
scikit-learn	Machine-learning library	Preprocessing, candidate models, metrics, K-Means, PCA.
Plotly Express	Interactive chart library	Confusion matrices, feature bars and cluster scatter plots.
joblib	Object serialization	Saved fitted model pipelines.
Docker	Container packaging	Build from Dockerfile; expose Streamlit on port 8501.
pytest + AppTest	Automated testing	Unit / integration tests and Streamlit page tests in tests/.

Frontend, processing, persistence

Frontend: Streamlit runs the page selected in the sidebar. Backend: the same Python process calls services for import, queries, feature creation and ML. Database: SQLAlchemy stores all eight datasets in one SQLite database. Model layer: scikit-learn pipelines are trained on demand and saved with joblib plus JSON metadata.

DEFAULT STARTUP

Database tables are initialized, but demo CSVs are not automatically imported and no models are automatically trained. Docker packages the same app and dependencies; it is not a separate service or database.

CSV schemas: roster and daily inputs

Every listed field is in the current validation schema. Examples are illustrative valid values, not model predictions.

players.csv - 4 required / 2 optional

Column	Meaning	Example	Req?	Used for
player_id	Stable player key	P007	Yes	Joins all tables
name	Player display name	Alex Rao	Yes	Roster / selectors
age	Age in years	24	Yes	Injury + performance
position	Playing position	Midfielder	Yes	Injury + performance
height	Height	180	No	Roster profile
weight	Body weight	76	No	Roster profile

training.csv - 7 required / 0 optional

Column	Meaning	Example	Req?	Used for
player_id	Player key	P007	Yes	Join
date	Training date	2026-07-01	Yes	Time window / sample date
duration_min	Session minutes	60	Yes	Training load
rpe	Perceived exertion, 0-10	7	Yes	Training load
distance	Total km	8.2	Yes	Validation / record
sprint_distance	Sprint metres	550	Yes	Sprint load 7d
high_speed_distance	High-speed metres	900	Yes	High-speed load 7d

recovery.csv - 8 required / 0 optional

Column	Meaning	Example	Req?	Used for
player_id	Player key	P007	Yes	Join
date	Recovery date	2026-07-01	Yes	Prior 7d window
sleep_hours	Sleep duration	7.5	Yes	7d mean, injury / performance
sleep_quality	Sleep score, 0-10	8	Yes	7d mean, injury / performance
hrv	Heart-rate variability	68	Yes	7d mean, injury / performance
resting_hr	Resting pulse	55	Yes	7d mean, injury / performance
soreness	Soreness score, 0-10	3	Yes	7d mean, injury / performance
stress	Stress score, 0-10	4	Yes	7d mean, injury / performance

CSV schemas: outcomes and matches

injuries.csv - 3 required / 2 optional

Column	Meaning	Example	Req?	Used for
player_id	Player key	P007	Yes	Join
injury_date	Outcome date	2026-07-08	Yes	Injury target / history
injury_occurred	Documented 0 or 1	1	Yes	Injury label / past count
injury_type	Recorded type	Muscle	No	Stored / display
days_missed	Absence days	6	No	Stored / display

matches.csv - 9 required / 0 optional

Column	Meaning	Example	Req?	Used for
player_id	Player key	P007	Yes	Join
match_date	Match date	2026-07-12	Yes	Target time / history
minutes_played	Minutes in match	90	Yes	Previous-5 sum
goals	Goals in match	1	Yes	Previous-5 sum
assists	Assists in match	0	Yes	Previous-5 sum
shots	Shot count	3	Yes	Stored / validated
passes_completed	Completed passes	37	Yes	Stored / display
key_passes	Key-pass count	2	Yes	Stored / display
rating	Observed match score, 0-10	7.4	Yes	Target; prior mean / slope / SD

health_events.csv - 3 required / 0 optional

Column	Meaning	Example	Req?	Used for
player_id	Player key	P007	Yes	Join
event_date	Outcome date	2026-07-08	Yes	Health target window
health_event	Documented 0 or 1	1	Yes	Health label only

KEY DISTINCTION

An uploaded CSV column is raw data. A rolling mean or count computed before a prediction date is a derived feature. injury_occurred, match rating and health_event are targets in their respective supervised tasks. Missing outcome rows do not become negative labels.

CSV schemas: health and medical tests

health.csv - 11 required / 6 optional

Column	Meaning	Example	Req?	Used for
player_id	Player key	P007	Yes	Join
date	Measurement date	2026-07-01	Yes	History / trends
resting_hr	Resting pulse	55	Yes	Health ML / trend / cluster
hrv	Heart-rate variability	68	Yes	Health ML / trend / cluster
sleep_hours	Sleep duration	7.5	Yes	Health ML / trend / cluster
sleep_quality	Sleep score	8	Yes	Health ML / trend / cluster
soreness	Soreness score	3	Yes	Health ML / trend / cluster
fatigue	Fatigue score	3	Yes	Health ML / trend / cluster
stress	Stress score	4	Yes	Health ML / trend / cluster
energy_level	Energy score	7	Yes	Health ML / trend / cluster
hydration_status	Hydration score	8	Yes	Health ML / trend / cluster
weight	Body weight	76	No	Latest ML / trend
body_fat_pct	Body-fat percent	14	No	Latest ML / trend
blood_pressure_sys	Systolic BP	120	No	Latest ML / trend
blood_pressure_dia	Diastolic BP	78	No	Latest ML / trend
oxygen_saturation	Oxygen saturation	98	No	Latest ML / trend
body_temperature	Temperature	36.7	No	Latest ML / trend

medical_tests.csv - 2 required / 11 optional

Column	Meaning	Example	Req?	Used for
player_id	Player key	P007	Yes	Join
test_date	Test date	2026-07-01	Yes	Prior-test lookup / trend
hemoglobin	Hemoglobin result	14.2	No	Health ML / trend / cluster
hematocrit	Hematocrit result	43	No	Health ML / trend
wbc_count	White-cell count	6.4	No	Health ML / trend
platelet_count	Platelet count	240	No	Health ML / trend
ferritin	Ferritin result	82	No	Health ML / trend / cluster
serum_iron	Serum iron result	94	No	Health ML / trend
vitamin_d	Vitamin D result	34	No	Health ML / trend / cluster
vitamin_b12	Vitamin B12 result	410	No	Health ML / trend
glucose	Glucose result	92	No	Health ML / trend
creatinine	Creatinine result	0.9	No	Health ML / trend
crp	CRP result	1.2	No	Health ML / trend / cluster

Medical analytes are optional and nullable. A missing test value is not an adverse finding; the fitted preprocessing pipeline imputes available model features and adds missing-value indicators.

Import, validation and database

What happens when a CSV is uploaded

Data page: choose a file; the app detects a dataset or accepts a selected type, offers column mapping, validates rows and previews valid/invalid results. Import occurs only on confirmation. A valid-only option can import passing rows; invalid rows are reported, never silently discarded. Manual entry follows the same schema checks.

Stage	Actual behavior
Schema	Required columns checked; supported optional columns accepted; column mapping and dataset detection offered.
Rows	ISO dates, finite numeric types, integers / booleans, missing required values, foreign player_id and duplicate player/date keys checked.
Ranges	Field-specific bounds; examples: RPE 0-10, sleep quality 0-10, HRV 1-300, resting HR 20-220, match rating 0-10.
Cross-fields	Sprint/high-speed metres cannot exceed total km converted to metres; goals cannot exceed shots; systolic BP must exceed diastolic BP.
Storage	SQLAlchemy transaction writes to SQLite; later queries create time-filtered frames for features and views.

One database, eight tables

Table	Purpose	Table	Purpose
players	Roster and player keys	training	Sessions / workload
recovery	Sleep and recovery	injuries	Documented injury outcomes
matches	Match history and ratings	health_records	Daily health values
medical_tests	Periodic lab measurements	health_events	Documented health outcomes

Connections: player_id links each child table to players. Each record date positions observations in time. The default SQLite file is data/football_ml.db; FOOTBALL_ML_DATABASE_URL can override the connection. Foreign keys are enabled, and initialization creates tables without loading demo rows.

LABEL COMPLETENESS

Injury and health negative windows require seven explicit daily 0 outcome records. An absent record is unknown, not automatically 0. This matters when explaining how supervised training examples are formed.

Feature engineering - exact formulas

Feature / formula	Small example	Used by
Session training load = duration_min x RPE	60 x 7 = 420	Injury; performance
Workload 7d / 28d = sum of prior session loads in [D-7,D) / [D-28,D)	420 + 300 = 720 in 7d	Injury; performance
Workload change % = $(7d / (28d/4) - 1) \times 100$; only if 28 days of history and 28d > 0	7d 2,020; 28d 7,600 -> +6.3%	Injury; performance
Sprint load 7d / high-speed load 7d = prior 7d sums of metres	250 + 300 = 550 m	Injury; performance
Recovery 7d mean = arithmetic mean of observed prior-7d values for each of six fields	Sleep 7 and 8 h -> 7.5 h	Injury; performance
Previous injuries = count of prior positive injury onsets	Two prior 1 records -> 2	Injury
Recent rating mean = mean of last up to five earlier matches	7.0, 8.0 -> 7.5	Performance
Rating trend = least-squares slope over previous match order; needs >=2	7.0, 8.0 -> +1/match	Performance
Rating consistency = population SD (ddof=0) of prior up-to-5 ratings; needs >=2	7.0, 8.0 -> 0.5	Performance
Recent minutes / goals / assists = sums over last up-to-5 earlier matches	90 + 80 -> 170 min	Performance

Health feature families

Prior 7 calendar days: for resting HR, HRV, sleep hours/quality, soreness, fatigue, stress, energy and hydration, the model creates a mean, least-squares trend per day and last-minus-first change. Trend/change need at least two observations. Example: HRV 70, 68, 66 on consecutive days yields mean 68, trend -2/day and change -4.

Other health: latest available value of weight, body fat, systolic/diastolic BP, oxygen saturation and temperature in the prior 7 days; count of 7d health observations; days since latest health record. Medical: for each of 11 tests, latest prior non-null result, latest-minus-previous change when two exist, and days since latest result. Example ferritin 90 then 80 -> change -10. These create 68 fixed health feature columns.

TIME RULE

Every model input is drawn from measurements strictly before its prediction date D. A 7-day outcome means dates D through D+6. Same-day or future measurements are excluded from features.

Injury and performance prediction

Injury prediction - next 7 days

Target: recorded new injury onset during D..D+6. A training example is anchored on the day after a training record. Explicit 0 injury records across all seven days define a negative example; missing follow-up is not a negative. Features: age/position; 7d and 28d workload, workload change, sprint/high-speed load; six recovery 7d means; prior injury count. Training date injury onsets are skipped.

Candidates: balanced Logistic Regression, balanced Random Forest Classifier (100 trees, min leaf 2) and Gradient Boosting Classifier. Chronological 60/20/20 date-group split with label-window purging. Winner maximizes 0.7 x validation recall + 0.3 x validation F1. It is refitted on train+validation and evaluated on untouched test data. At least 40 labelled windows, 5 positive windows and 5 recorded injury events are required, plus split/class and feature-coverage checks.

Predicted injury probability	Displayed band
0% to <40%	LOW
40% to <70%	MEDIUM
70% to 100%	HIGH

The probability is the fitted classifier's predict_proba estimate for a documented injury event in that seven-day window. The band is a display rule applied afterward; it is not a separate diagnosis or model. Model Analysis displays global feature importance; the prediction page also shows model feature information.

Performance prediction - next match rating

Target: the numeric rating in a later match record, on its 0-10 source scale. Features are age/position, the same prior workload/recovery features, and earlier-match rating mean, trend, consistency plus minutes/goals/assists sums from up to five earlier matches. The target match's statistics are never fed in as inputs. A first match with no prior match history produces no training sample.

Candidates: Linear Regression, Random Forest Regressor (100 trees, min leaf 2) and Gradient Boosting Regressor. The same chronological split/purge framework is used. Winner has the lowest validation MAE; train+validation refit precedes test evaluation. Readiness requires at least 35 examples, variable train ratings and feature coverage. The result is the raw predicted rating; the UI does not assign Poor/Good/Excellent categories or clip the estimate to 0-10.

PREPROCESSING / ARTIFACTS

Both tasks use median numeric imputation, missingness indicators and scaling; position is one-hot encoded with unknown categories ignored. Each saved joblib pipeline and JSON metadata lives under data/model_artifacts/ with a task-specific fingerprint, feature list and metrics.

Monitoring, trends and clustering

A. Health Monitoring - supervised ML

Inputs: health.csv plus optional medical_tests.csv history. Target: explicit health_event=1 during D..D+6; a 0 window requires seven explicit daily 0 event records. Sample date is the day after a health record. At prediction time, at least one health observation in the prior 7 days is required; medical tests are optional. The model estimates a documented event risk, not a condition or diagnosis.

Training: 68 fixed features; median imputation, missing indicators and scaling. Logistic Regression, Random Forest Classifier and Gradient Boosting Classifier are compared. Distinct-date 60/20/20 chronological train/validation/test split; overlapping outcome windows are purged at boundaries. Winner maximizes $0.7 \times \text{validation recall} + 0.3 \times \text{validation F1}$, then refits on train+validation and is tested. At least 40 labelled windows, 5 documented events and class support are required.

Health event probability	Displayed risk
0% to <40%	LOW
40% to <70%	MEDIUM
70% to 100%	HIGH

The selected player's probability comes from predict_proba. The displayed top features use a local median-replacement probability comparison; they are model associations, not causes. Health artifacts are separate: data/model_artifacts/health/*.joblib and matching JSON metadata.

B. Health Trends - descriptive statistics

Selected player charts cover heart (resting HR/HRV), sleep, recovery, hydration, body/vitals and available medical tests. Each series shows latest value, rolling 7-calendar-day health average or 90-day medical average, previous-to-latest % change = $(\text{latest} / \text{previous} - 1) \times 100$ when previous is nonzero, and a direction from the calendar-day regression slope of the last up to seven observed points (rising, declining, stable; insufficient with <2). These are calculations and charts, not ML predictions.

C. Health Clustering - unsupervised ML

K-Means groups players using the nine recent health means plus latest hemoglobin, ferritin, vitamin D and CRP when sufficiently available and varying. Features must be observed in at least half the roster; remaining gaps are median-imputed and features standardized. The user chooses 2-5 clusters (default 3); K-Means uses 10 starts and fixed seed 42. Assignments are labels 1..K, with raw-feature cluster means and descriptions based on largest standardized deviations from roster averages.

A 2D PCA projection visualizes the groups only; K-Means itself uses the prepared feature space. Cluster numbers carry no health ranking. The code does not compute a silhouette score.

Models, evaluation and selection

Model	Type / use	Plain-language idea
Logistic Regression	Classifier: injury, health	Linear combination mapped to event probability.
Random Forest Classifier	Classifier: injury, health	Average of many decision trees.
Gradient Boosting Classifier	Classifier: injury, health	Trees added sequentially to reduce errors.
Linear Regression	Regressor: performance	Linear estimate of numeric match rating.
Random Forest Regressor	Regressor: performance	Average predictions of many trees.
Gradient Boosting Regressor	Regressor: performance	Successive trees improve numeric estimates.
K-Means	Clustering: health	Assign profiles to nearest learned centroid.
PCA	Visualization only	Projects cluster data into two chart axes.

Classification predicts an event probability/class; regression predicts a number; clustering finds groups without event labels. XGBoost and neural networks are not implemented.

Metric quick reference

Metric	Meaning / range / direction
Accuracy	$(TP+TN)/N$; 0-1; higher, but can hide rare events.
Precision	$TP/(TP+FP)$; 0-1; higher means fewer false alarms among positives.
Recall	$TP/(TP+FN)$; 0-1; higher means fewer missed events.
F1	$2PR/(P+R)$; 0-1; higher balances precision and recall.
ROC-AUC	Ranking across thresholds; 0-1; higher, 0.5 is chance-like; absent if test has one class.
Confusion matrix	Counts TN, FP, FN, TP; inspect error trade-off, not a single score.
MAE	Mean actual-predicted in rating points; ≥ 0 , lower better.
RMSE	$\sqrt{\text{mean squared error}}$, rating points; ≥ 0 , lower better; penalizes large errors.
R2	$1 - SSE/SST$; ≤ 1 , 1 ideal, can be negative; absent when actual ratings do not vary.
Cluster sizes / profiles	Counts and averages; descriptive, with no universal higher/lower rule.

Interpret actual validation/test values against task prevalence, baseline rating error and train-test gaps. The code defines no universal pass mark. Health and injury select by 0.7 recall + 0.3 F1; performance selects lowest MAE. The newest test dates do not select the winner.

Pages, controls and behind-the-screen flow

Page (sidebar order)	What the user sees / uses	Data, calculation and ML
Dashboard	Eight table counts; model readiness; small injury, performance and health summaries.	Queries DB and saved models; current eligible predictions where available.
Data	CSV uploader, dataset choice/mapping, validation preview, import confirmation, manual entry, records, template/export.	Validation service -> SQLAlchemy transaction -> eight tables; no model training.
Players	Roster/selector, longitudinal training/recovery/match/injury sections, charts, snapshot and predictions.	Queries selected player, feature snapshot, saved predictions if eligible.
ML Predictions	Prediction-date selector, injury probability/band table, next-match rating and model feature information.	Loads artifact; builds pre-date features; calls classifier/regressor.
Model Analysis	Injury and Performance tabs, readiness, Train buttons, validation candidates, held-out test metrics and charts.	Builds labels/features; chronological split; fit/compare/refit/save.
Health	Monitoring, Trends and Clustering internal tabs with respective train/predict, charts and run controls.	Health classifier, statistical series and K-Means/PCA respectively.

Button-to-result: example sequence

Train Injury Model: Streamlit button -> service retrieves dated rows -> feature builder creates pre-date inputs and documented labels -> chronological train/validation/test with overlap purge -> three candidates fit -> validation score selects winner -> winner refits -> test metrics produced -> pipeline and JSON metadata saved -> comparison/metrics shown in Model Analysis.

Predict Health Risk: Streamlit player/date controls -> saved health pipeline loaded -> earlier health/medical rows queried -> 68 features assembled -> pipeline preprocesses and predicts event probability -> 40%/70% display band and local feature association list shown. The interface refuses a backdated forecast earlier than the model's fitted-through date.

UI <-> BACKEND

USER -> STREAMLIT VIEW -> PYTHON SERVICE -> SQLALCHEMY / SQLITE -> FEATURE ENGINEERING -> SAVED SCIKIT-LEARN PIPELINE -> RESULT -> STREAMLIT TABLE OR CHART

Worked example and quick answers

This page separates arithmetic features from probabilities learned by a fitted model.

End-to-end fictional example (using implemented logic)

P007 has a 60-minute training session at RPE 7, so session load = $60 \times 7 = 420$. Other prior-seven-day sessions contribute 1,600, giving workload_7d = 2,020. If prior-28-day load is 7,600 and full history exists, the four-week weekly baseline is $7,600/4 = 1,900$ and workload change = $(2,020/1,900 - 1) \times 100 = 6.3\%$. Earlier recovery values and prior injuries become other inputs. A fitted classifier, not this arithmetic, supplies the probability; only then does the UI apply LOW (<40%), MEDIUM (40-<70%) or HIGH (>=70%).

Scores and source ranges

Item	Verified code behavior
Injury / health bands	LOW $p < 0.40$; MEDIUM $0.40 \leq p < 0.70$; HIGH $p \geq 0.70$.
Performance rating	Source rating validated 0-10; output is raw predicted rating, with no category bands.
Questionnaire scales	RPE, sleep quality, soreness, stress, fatigue, energy, hydration: 0-10 in relevant datasets.
Other key bounds	Sleep 0-24 hours; HRV 1-300; resting HR 20-220; age 10-70; match minutes 1-130.
Readiness	Injury/health ≥ 40 labelled windows, ≥ 5 positive windows and events; performance ≥ 35 samples; further split/coverage checks.

Short viva answers

Question	Answer
Why Streamlit?	Builds interactive Python pages for uploads, selectors, metrics and charts.
Why SQLite?	Simple persistent local relational storage for the eight linked tables.
Why these ML models?	Interpretable baseline plus tree ensembles; validation chooses for each task.
Feature engineering?	Convert dated raw records into historical windows, changes and counts.
Data leakage?	Using future information in past inputs; strict pre-date features, date split and purge prevent it.
How do predictions work?	Saved preprocessing + selected fitted model transform pre-date features into a probability/rating.
Best model?	Injury/health: highest 0.7 recall + 0.3 F1; performance: lowest validation MAE.
Health Monitoring?	Supervised future documented health-event risk, using health and medical history.
Health Trends?	Statistical summaries, rolling values, changes and charts; no separate ML model.
Health Clustering?	K-Means groups similar player profiles; PCA is for plotting only.
Docker?	Packages the app/dependencies and starts Streamlit on port 8501.

Full flow: explicit CSV upload -> validated SQLite rows -> time-safe features/labels -> train/validation/test -> saved task artifact -> prediction or descriptive analysis -> Streamlit display. Demo files live in demo_data_final/ and are not auto-loaded.